Voice Mode MisoTTS Latency Report

What we tested, why MisoTTS 8B missed the live voice target, and what the practical fix should be.

LearningAI / Tutor - prepared June 7, 2026

Finding	Fresh MisoTTS 8B generation was seconds to minutes, not under 200 ms.
Local broker result	30 local trials: p95 ConversationText 0.704 ms, about 704 us.
Decision	Keep Miso for read-aloud or async narration; use realtime/browser TTS for live first speech.

Executive takeaway	The issue is not a small HTTP optimization problem. Public MisoTTS 8B generated whole utterances too slowly and did not expose streaming first audio. The reliable local fix is to keep Miso out of the live first-audio path until a true streaming model is proven.

Date: 2026-06-07

Chapter 1: What We Tried To Build

The target architecture was a low-latency voice tutor loop:

- Deepgram handles speech-to-text from the microphone.
- GPT-4o-mini, through OpenRouter, handles the foreground teaching response.
- GPT-5.5 handles asynchronous background work such as web search, PDFs, code analysis, and deeper tool jobs.
- MisoTTS 8B runs on a rented GPU machine and speaks the assistant response.
- FastAPI wraps the GPU-side TTS service.
- Tutor's local server connects the browser, foreground model, background jobs, STT, and TTS through `/api/voice-broker`.

The product goal was strict: first usable spoken response under 200-300 ms, with a conversational socket open between the learner and the voice system.

Chapter 2: GPU Instance And Remote Setup

The tested Vast instance was reached through:

```bash
ssh -p 19691 root@175.155.64.137 -L 8080:localhost:8080
```

The actual working local tunnel mapped local port 8080 to the remote Miso wrapper on port 8090:

```bash
ssh -fN -o ExitOnForwardFailure=yes -p 19691 root@175.155.64.137 -L 8080:localhost:8090
```

Observed machine class:

- GPU: 1x RTX 4090, 24 GB advertised class, enough VRAM for the public MisoTTS 8B bfloat16 path.
- OS: Ubuntu 24.x class remote image.
- Remote service: `/workspace/miso-service/misotts_api_server.py`
- Miso repo: `/workspace/MisoTTS`
- Hugging Face cache: `/workspace/hf`
- Miso model snapshot: `/workspace/hf/hub/models--MisoLabs--MisoTTS/snapshots/ef6b096cc35d3cde6aa0721013648416c14c36b2/model.safetensors`

The Meta tokenizer repo was gated, so the wrapper used:

```bash
MISO_TTS_TOKENIZER_MODEL=unsloth/Llama-3.2-1B
```

The tokenizer was checked for compatibility at the practical level needed here: BOS/EOS and vocabulary behavior matched the expected Llama 3.2 tokenizer family.

Chapter 3: What We Implemented Locally

The local app gained a custom voice broker route:

- WebSocket path: `/api/voice-broker`
- Feature flag: `VITE_VOICE_BROKER_MODE=custom`
- Browser speech delegation flag: `VITE_VOICE_BROKER_BROWSER_TTS=true`
- Live TTS deadline: `VOICE_BROKER_TTS_DEADLINE_MS=180`
- Optional Miso endpoint: `MISO_TTS_API_URL=http://127.0.0.1:8080`

The Miso wrapper gained:

- `/health`
- `/v1/audio/speech`
- `/v1/audio/prewarm`
- disk WAV cache under `MISO_TTS_CACHE_DIR`
- stage timing logs
- tokenizer source override
- `max_audio_length_ms` minimum lowered to 80 ms for probing

The browser voice path gained:

- `ConversationText` playback through `window.speechSynthesis`
- immediate cancellation on barge-in
- immediate cancellation on voice stop
- local broker auth field `browserTts: true`
- no fresh Miso generation in the live first-audio path when browser TTS is active

Chapter 4: What The Measurements Showed

The important finding is that the public MisoTTS 8B wrapper is not a true sub-200 ms live TTS path for arbitrary fresh text on this setup.

Unit note:

- `s` means seconds.
- `ms` means milliseconds.
- `us` means microseconds.
- `1 ms = 1,000 us`.
- Values below `1.000 ms` are sub-millisecond local measurements. For example, `0.324 ms` is about `324 us`.

Observed Miso timings:

Case	Result
Cold load plus first generation	about 89.59 s total; generator load about 83.70 s
Warm fresh generation	about 5-6 s in early simple smoke tests
Later fresh generation probes	16.48 s, 18.63 s, 61.44 s, and 198.59 s
Cached WAV on GPU box	effectively instant in wrapper logs
Cached WAV through SSH tunnel	often around 1.2-2.6 s from the Mac

The remote log showed fresh generation waits behind a single generation lock. That explains the user-visible symptom: "not reading at times." It was not just a browser playback bug; the server could be waiting on whole-utterance generation.

The issue was therefore two separate latency problems:

Issue	What we observed	Why it matters
Fresh Miso generation was seconds to minutes	16.48 s, 18.63 s, 61.44 s, and 198.59 s fresh generations	This cannot satisfy a 200 ms first-audio target.
The Miso wrapper returns whole WAV files	Audio is emitted only after token generation, decode, watermark, resample, and WAV encoding	The browser gets no early audio chunk to play.
A single generation lock serialized work	Later calls waited behind slow calls	The system can appear to stop reading.
SSH tunnel and network path added delay	Cached remote responses were instant on the GPU box but around 1.2-2.6 s from the Mac	Even cached audio was not reliable as the live first-audio path through that tunnel.
Cached audio is not the real solution	Cached acknowledgements can be fast, but arbitrary new responses still miss the target	It can hide the problem for "Okay" but not solve tutoring speech.

The public Miso source confirmed the architecture:

1. tokenize text,
2. generate audio tokens frame by frame,
3. decode the full token stack into audio,
4. watermark and resample,
5. encode and return a full WAV.

There was no exposed streaming API that could emit the first audio chunk before whole-utterance completion.

Chapter 5: Thirty Local Latency Trials

After moving live speech to browser TTS delegation, we ran 30 local websocket trials on the Mac against:

```text
ws://127.0.0.1:3001/api/voice-broker?language=en
```

Each trial opened the broker, sent `voice_auth` with `browserTts: true`, waited for `VoiceBrokerReady`, injected a text turn, and measured:

- `ConversationText` latency from injection
- `AgentFinishedSpeaking` latency from injection

No OpenRouter key was used in this latency test, so the foreground reply was the provider-safe local staged response. This measures the local broker loop after removing Miso from the blocking path, not full Deepgram STT plus GPT-4o-mini plus TTS.

Trial	ConversationText ms	AgentFinished ms
1	0.449	0.472
2	0.736	0.871
3	0.286	0.304
4	0.465	0.470
5	0.333	0.343
6	0.494	0.550
7	0.437	0.472
8	0.704	0.722
9	0.301	0.385
10	0.228	0.278
11	0.889	0.905
12	0.388	0.397
13	0.534	0.576
14	0.360	0.396
15	0.324	0.331
16	0.226	0.236
17	0.254	0.284
18	0.264	0.271
19	0.320	0.347
20	0.611	0.673
21	0.139	0.154
22	0.106	0.108

Trial	ConversationText ms	AgentFinished ms
23	0.099	0.110
24	0.133	0.136
25	0.142	0.164
26	0.090	0.176
27	0.125	0.142
28	0.326	0.497
29	0.431	0.480
30	0.416	0.424

Summary:

Metric	ConversationText	AgentFinished
Count	30	30
Min	0.090 ms	0.108 ms
p50	0.324 ms	0.347 ms
p90	0.611 ms	0.673 ms
p95	0.704 ms	0.722 ms
Max	0.889 ms	0.905 ms
Average	0.354 ms	0.389 ms

The complete 30-trial run took 78.715 ms wall time.

The local trial values are milliseconds. Because they are below `1.000 ms`, they can also be read as approximate microseconds:

Metric	ConversationText	AgentFinished
p50	0.324 ms, about 324 us	0.347 ms, about 347 us
p95	0.704 ms, about 704 us	0.722 ms, about 722 us
Max	0.889 ms, about 889 us	0.905 ms, about 905 us

These numbers are intentionally only the local broker loop after browser TTS delegation. They do not claim the full future stack is sub-millisecond. The full live stack still needs separate measurement for microphone capture, Deepgram STT, GPT-4o-mini response generation, browser speech start, and background job insertion.

Chapter 6: Decision

The decision is to delete the current GPU instance for now and not treat MisoTTS 8B as the live first-audio path.

MisoTTS 8B can remain useful for:

- non-realtime read-aloud,
- pre-generated chapter audio,
- cached acknowledgement clips,
- future async high-quality narration,
- experiments if Miso publishes a true streaming or server-optimized path.

It should not be the live voice loop renderer unless a future model/server proves fresh arbitrary text under the deadline.

The live voice path should be:

```text
Browser mic -> Deepgram STT -> Tutor voice broker -> GPT-4o-mini foreground text
                        -> browser/local realtime TTS for first speech
                        -> GPT-5.5 background jobs asynchronously
```

Miso can be attached only after it stops blocking the foreground loop.

Chapter 7: Possible Fixes

The practical fix we applied locally was architectural:

1. Keep `/api/voice-broker` as the realtime coordinator.
2. Send assistant `ConversationText` immediately.
3. Let the browser speak that text with `window.speechSynthesis`.
4. Keep MisoTTS out of the live first-audio path.
5. Keep MisoTTS for read-aloud, cached clips, or async narration only.
6. Record the latency and decision in this book so the failed GPU path is not repeated blindly.

Possible future fixes if we want a better voice than browser TTS:

Fix candidate	What it would need to prove	Risk
True streaming TTS model	First audio chunk under 200 ms for uncached arbitrary text, with stable p95	Quality or deployment complexity may be worse.
Managed realtime TTS API	Low first-byte audio latency from our region, barge-in/cancel support, predictable cost	Vendor dependency and API cost.
Smaller local TTS model	Runs on smaller GPU/CPU and streams chunks immediately	Voice quality may be weaker than Miso.
Prewarm/cache common phrases	Fast acknowledgements and common tutor phrases	Does not solve arbitrary fresh responses.
Rewrite Miso server for streaming	Decode and send audio chunks before whole utterance completion	Public Miso path did not expose this; may require deep model/vocoder work and still may not hit p95.
Move app/server/GPU closer together	Reduce network and SSH tunnel overhead	Does not fix multi-second fresh generation.

The main issue to fix is not "make the HTTP route faster." The main issue is that whole-utterance Miso generation is too slow and non-streaming for realtime tutoring. The replacement must stream early audio, cancel cleanly, and keep the foreground websocket free.

Chapter 8: What To Look For Next

The replacement live TTS model or service must prove:

- first audio under 200 ms for uncached arbitrary text,
- streaming output rather than whole-WAV completion,
- cancellation/barge-in support,
- stable p95 latency, not just one fast demo,
- no single global generation lock that blocks all sessions,
- acceptable voice quality for tutoring,
- predictable deployment on a small GPU or CPU instance.

Good candidates to test next are true streaming TTS engines or managed realtime speech APIs. A local model is only acceptable if it can stream chunks quickly on the target hardware.

Chapter 9: What Remains Local

The local repo now contains the important reusable work:

- Tutor voice broker route in `server.ts`
- browser realtime TTS delegation in `src/components/ChatPanel.tsx`
- Miso wrapper script in `scripts/misotts_api_server.py`
- environment flags in `.env.example`
- architecture notes in this book

Deleting the Vast instance loses the remote cache/model files and running service, but not the local integration work.